PROJECT REPORT

Dynamic AI Agent Orchestration Platform

Setup, demonstration, architecture and design rationale

Part 1 Project overview — setup, running it, demonstration scenarios, API, assumptions and limitations

Part 2 Architecture — agent design, routing, LangGraph and LangChain usage, MCP integration, memory and persistence

> Part 1 covers what the platform does and how to run it. Part 2 explains why it is built the way it is, and defends the decisions behind it. A companion document, `docs/codebase-guide.pdf`, walks through the code itself for a reader coming to it for the first time.

Contents

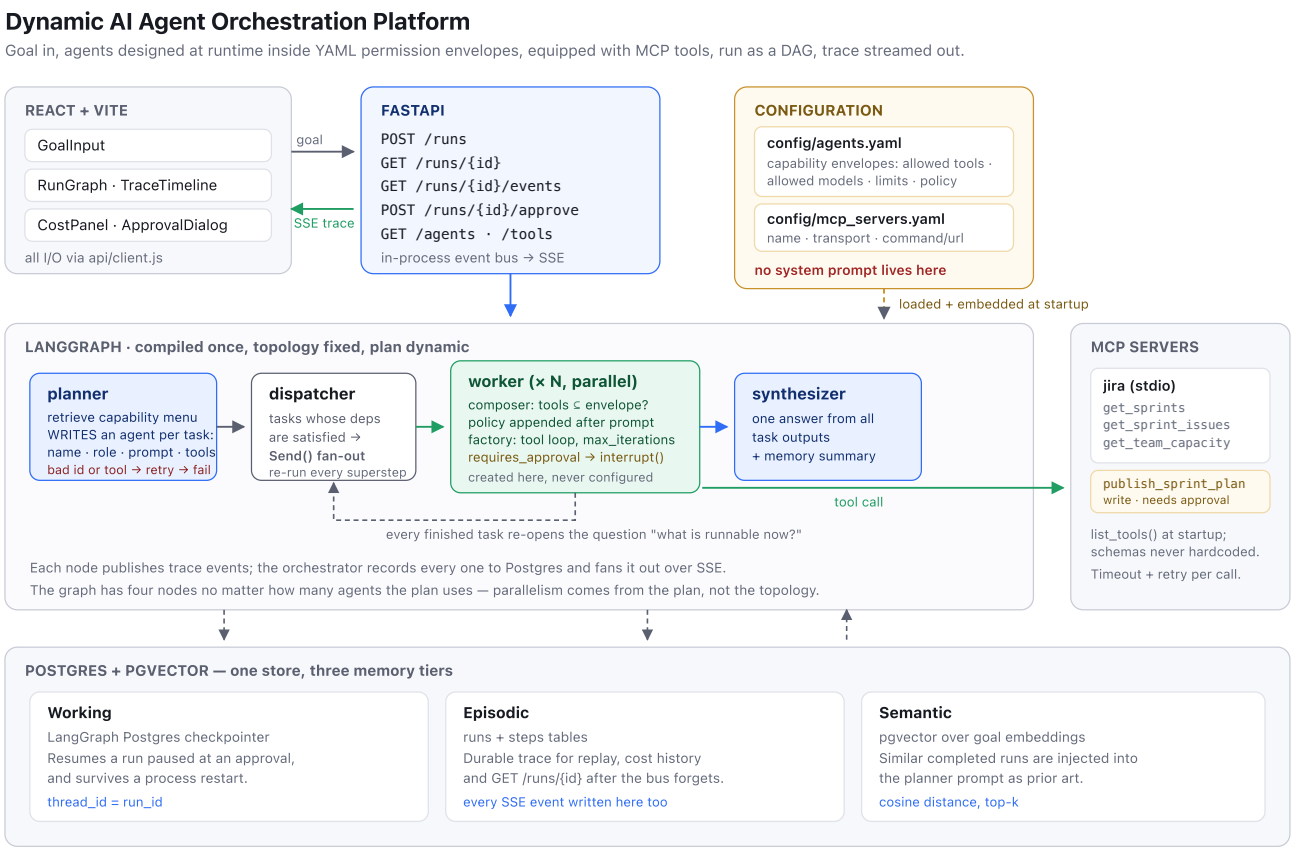
PART 1

Project overview

Submit a natural-language goal. The platform **designs the specialist agents for it at runtime** — writing each agent's name, role and system prompt, and choosing its tools — inside permission envelopes declared in YAML, runs them as a DAG, and returns a synthesized answer with a full execution trace.

> **"Predict the velocity for our next sprint based on previous Jira sprints."**
>
> → the planner creates a `velocity_forecaster` agent under the `jira_reporting` envelope, writes it a five-step method ("call `jira.get_sprints` … show the arithmetic explicitly… only call it a trend if the movement is sustained"), grants it the two read tools it asked for, and streams the run to the browser.

No agent's prompt exists as a fixed string in this repository. There is no `JiraAgent` class and no agent record in YAML. `agents.yaml` declares *capabilities* — what a class of agent may touch and the policy it must obey — and the planner writes the agent. A generated agent can narrow its envelope, never widen it, which is what keeps the approval gate and the tool permissions meaningful.



Full design rationale: **Part 2**.

New to the codebase? **docs/codebase-guide.pdf** traces one real request from the browser to the answer, explains every module, and assumes no prior knowledge of MCP, LangGraph or embeddings.

Quick start

With Docker (recommended)

```
cp .env.example .env
# Put a real key in ANTHROPIC_API_KEY — everything else has working defaults.
# OPENAI_API_KEY is optional and used only for embeddings; without it retrieval
# falls back to a local lexical embedder that matches words rather than meaning.

docker compose up --build
```

`.env` is passed wholesale to the backend container, so **any setting you add there reaches the app without editing** `docker-compose.yml`. Two deliberate exceptions: `DATABASE_URL` is overridden to `postgres:5432` (a localhost URL would point the container at itself), and the database and frontend containers get only the two or three values they need rather than the API key. The file is optional — with no `.env`, everything falls back to the defaults in `backend/app/settings.py`.

UI	http://localhost:5173
API docs (Swagger)	http://localhost:8000/docs
Health	http://localhost:8000/health

The mock Jira MCP server runs as a stdio subprocess of the backend container, so there is no fourth service to start.

Without Docker

```
docker compose up -d postgres          # Postgres + pgvector, published on 5433

python3.12 -m venv .venv && source .venv/bin/activate
pip install -e "backend[dev]"
cp .env.example .env                   # DATABASE_URL already points at localhost:5433

cd backend && uvicorn app.main:app --reload    # :8000
cd frontend && npm install && npm run dev      # :5173
```

Postgres is published on **5433**, not 5432, so it cannot collide with a Postgres you may already be running.

Running the tests

Tests stub the LLM and **need no API key**. They start the real mock MCP server over stdio, so tool discovery and invocation are genuinely exercised.

```
cd backend && pytest          # 81 tests, ~12s
docker compose exec backend pytest      # or inside the container
```

Demonstration scenarios

Each is a one-click example in the UI.

#	Prompt	What it demonstrates
1	*Predict the velocity for our next sprint based on previous Jira sprints.*	The core scenario: multi-agent plan, MCP tool use, synthesis
2	*How many story points did we complete in the last three sprints, and what was our completion rate?*	**Small goals get small plans** — one agent, not four
3	*Two independent things about our next sprint: a velocity forecast from sprint history, and the team capacity available. Then review the combined picture for delivery risk.*	**Parallel fan-out** — the forecast and capacity tasks are dispatched in one superstep, then converge on the reviewer
4	*Work out our next sprint commitment and publish it to the Jira board.*	**Human approval** — the run pauses before the write tool and waits for you

Scenario 3 is worth watching in the trace: both branches report `task.started` before either one's tool call returns. Phrasing matters — ask instead to *"forecast velocity and **adjust it** for availability"* and the planner correctly produces a sequential chain, because the second task then genuinely depends on the first. The plan is a real decision, not a fixed pipeline.

Scenario 4 is the one to try deliberately: `jira_writeback` is the only envelope that can write to Jira, and it is marked `requires_approval: true`. The graph interrupts *before* the agent runs and checkpoints to Postgres. The approval dialog shows **the generated prompt that is about to run**, since that is what you are actually approving. Decline it and the write never happens; the run still completes and says what was skipped.

Watching agents get created

Run the same goal twice and compare `plan[].agent.system_prompt` in `GET /runs/{id}` — the agents are written fresh each time. Then edit `backend/config/agents.yaml` and watch the envelope constrain them: narrow `jira_reporting`'s `allowed_tool_selectors` to just `jira.get_sprints` and the next run's agent can no longer ask for issue-level data, however it is prompted. The "Registry" panel in the UI shows the envelopes the planner is designing against.

API

Interactive docs and the OpenAPI schema are generated at `/docs` and `/openapi.json`.

Method	Path	Purpose
`POST`	`/runs`	Submit a goal; returns `run_id` immediately (202)
`GET`	`/runs/{id}`	Final state, plan, answer, cost and durable trace
`GET`	`/runs/{id}/events`	SSE trace stream
`POST`	`/runs/{id}/approve`	Resume an interrupted run
`GET`	`/agents`	The loaded capability registry, ceilings resolved to real tools
`GET`	`/tools`	Tools discovered from MCP servers at startup

```
RUN=$(curl -s -X POST localhost:8000/runs -H 'content-type: application/json' \
  -d '{"goal":"Predict the velocity for our next sprint."}' | jq -r .run_id)

curl -N localhost:8000/runs/$RUN/events      # live trace
curl -s localhost:8000/runs/$RUN | jq        # final state
```

The SSE stream **replays everything already published** before going live, so attaching after `POST /runs` — or after the run has already finished — still renders the full trace.

Adding a capability (the whole point)

You do not add agents — the planner writes those. You add an *envelope*: a new class of thing agents are allowed to do. Append to `backend/config/agents.yaml` and restart. No Python.

```
  - id: release_communication
    description: >-
      Drafts release notes and changelogs from the issues completed in a sprint,
      grouped by change type and written for a non-technical audience.
    allowed_tool_selectors: ["jira.get_sprint_issues"]
    allowed_models: ["claude-sonnet-5", "claude-haiku-4-5"]
    default_model: claude-sonnet-5
    max_iterations_limit: 4
    requires_approval: false
    policy: |
      Never describe work that is not in the issue list you were given.
```

Note what is **not** there: no `system_prompt`. Putting one in is rejected at startup — that is the thing this design removed.

`description` is embedded and matched against the user's goal, so **write it as a description of capability using the words a user would use**. This is not cosmetic: an early version of the velocity envelope ranked poorly for velocity goals because its text never contained the word "velocity".

`policy` is the escape hatch for a rule that must hold no matter what the planner writes. It is appended *after* the generated prompt under an explicit override header, because later instructions carry more weight than earlier ones.

Adding an integration is the same kind of edit to `backend/config/mcp_servers.yaml`.

Technology choices

Choice	Why
FastAPI	Async, native SSE, and OpenAPI docs for free — a required deliverable
LangGraph	`Send` for runtime fan-out, `interrupt()` for durable human approval, Postgres checkpointing. Used for the runtime only — no prebuilt agents
LangChain	Only `langchain-mcp-adapters` and `ChatAnthropic`. No chains, no agents, no memory
Postgres + pgvector	One store for all three memory tiers. A second datastore would need to earn its place
React + Vite	Fast dev server, no framework overhead
React Flow	Renders the run DAG without hand-drawn SVG
Claude	`claude-opus-5` plans and synthesizes; workers run `claude-sonnet-5`. Each capability declares the models it permits, and the planner picks one from that list

Not used, deliberately:

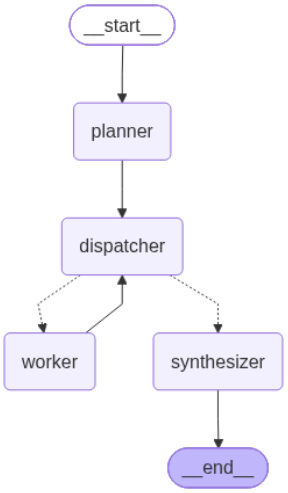
- **LangFlow** — its flow definition lives in GUI state, which contradicts the premise that routing is version-controlled configuration decided at runtime.
- **Redis** — a single API process has one publisher and a few SSE subscribers. There is nothing for a broker to do, and the durable trace is in Postgres.

How it works, briefly

1. **Startup** loads `agents.yaml` (embedding each capability `description`), opens a session to every server in `mcp_servers.yaml` and calls `list_tools()`, prepares Postgres, and compiles the graph. Tool schemas are never hardcoded.
2. **Planning** embeds the goal, retrieves a menu of candidate capabilities, relevant tools and similar past runs, then makes one structured-output call that **designs an agent per task** — name, role, system prompt, tools. Two checks follow: `capability_id` against the registry, then every synthesized agent is dry-run through the envelope. Either failure is retried once with the error fed back, then fails the run. Checking here means an escalation costs a retry rather than a half-finished run.
3. **Execution** dispatches every task whose dependencies are satisfied, in parallel, via LangGraph `Send`. The graph has four nodes no matter how many agents run; parallelism comes from the plan.
4. **Each worker** realizes its agent against the envelope again — the worker does not trust the plan it was handed — appends the capability's policy to the generated prompt, binds only the tools that were granted, runs a tool loop bounded by `max_iterations`, and emits a trace event per tool call.
5. **Synthesis** produces the answer plus a summary that is embedded for future planning.

The compiled graph

The whole topology, whatever the plan turns out to be:



The compiled graph: four nodes, whatever the plan asks for. The dotted edges out of `dispatcher` are the conditional ones.

The dotted edges out of `dispatcher` are the conditional ones — that is the scheduler, and it is the only reason a plan of any shape runs with the right parallelism. `worker → dispatcher` is a genuine cycle: each finished task re-opens the question of what is runnable now. See **Part 2 §3**.

Assumptions

- **Jira is mocked.** No credentials were available, so `mcp_servers/jira_mock/` serves fixture data over real MCP/stdio. Swapping in a real server is a config edit; see the commented block in `mcp_servers.yaml`.
- **A rolling average is the forecast.** The forecasting agent computes a mean of recent completed points and says that is what it is. A real forecasting model is out of scope, and dressing a mean up as one would be worse than saying so.
- **Single tenant, single process.** No auth, one shared set of MCP sessions.
- **The demo fixture is synthetic** — eight closed sprints with plausible committed/completed spreads.

Known limitations

- **Generated prompts vary between runs.** This is the direct cost of creating agents at runtime: the same goal can produce differently-worded agents, so output is less repeatable than a hand-tuned prompt would be, and a bad generation is possible. Two things bound it — the capability `policy` carries the rules that must hold regardless of wording, and every run records the exact prompt that ran in `plan[].agent.system_prompt` and in the `task.started` event, so any run can be audited or reproduced after the fact. Pinning a known-good generated prompt per capability would be the next step if determinism mattered more than adaptability.
- **Planning costs more than it used to.** The planner writes prompts now, so it emits far more output tokens per plan than when it only picked ids.
- **The live event bus is in-process.** It does not fan out across replicas and does not survive a restart. The durable trace in `steps` does, and `GET /runs/{id}` serves it.

- **Retrieval degrades without an OpenAI key.** Embeddings come from `text-embedding-3-small`; with no key configured the platform falls back to a local lexical vectorizer that matches shared words rather than meaning. On paraphrased goals that costs real accuracy — measured over a set of reworded queries, top-1 retrieval is **5/6 hosted against 3/6 on the fallback**. Everything still runs; it just retrieves worse. See **Part 2 §5**.
- **A capability's `description` is retrieval surface.** A description written about *mechanism* rather than *capability* retrieves badly whatever the model: rewording one of the four moved its score for a matching goal from 0.25 to 0.53.
- **Changing the embedding model invalidates stored vectors.** They live in a different space, so past runs rank arbitrarily until re-run. Nothing crashes if the width matches. `docker compose down -v` clears them.
- **No authentication**, and any caller can approve a paused run.
- **Schema is created with `create_all`** at boot, not migrations.
- **Planner quality is untested.** There is no eval set scoring whether the planner picks the right capabilities or writes good prompts; the tests pin the *contract* (it cannot invent an id, it cannot escape an envelope, it cannot emit a cycle), not the judgement. Prompt quality is now part of what the planner is responsible for, which makes this gap larger than it was.
- **MCP sessions are process-wide**, so tools cannot carry per-user credentials.
- **No `task.completed` event**, by contract. The client derives per-task completion; see **Part 2 §7**.

What I would improve for production

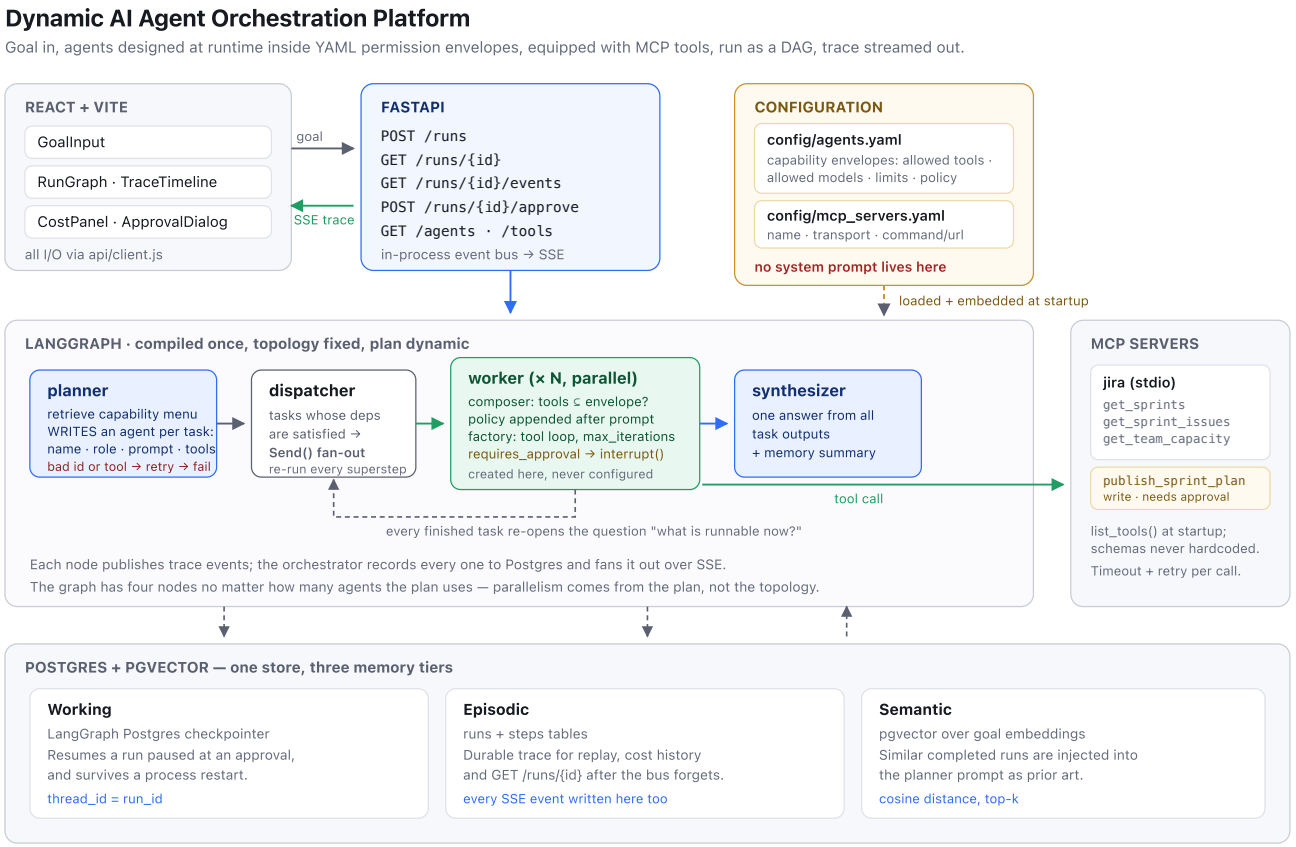
Hosted embeddings behind the existing seam; Postgres `LISTEN/NOTIFY` before reaching for a broker; Alembic migrations; per-tenant auth with MCP credentials scoped per user rather than per process; a labelled goal→plan eval set so planner changes can be measured; and recording *who* approved a sensitive run.

Repository layout

```
backend/
  config/agents.yaml         the capability registry — add envelopes here
  config/mcp_servers.yaml    the MCP server registry — add integrations here
  app/
    agents/                  capability + agent models, registry, composer, factory
    graph/                   state & plan contract, planner, worker, synthesizer, build
    mcp/                     session manager (timeouts, retries), tool index & search
    memory/                  engine, tables, pgvector retrieval
    api/                     routes and response schemas
    orchestrator.py          drives one run: events out, state persisted
    events.py  costs.py  embeddings.py  chat_models.py  settings.py
  tests/                     registry, factory, planner, end-to-end
mcp_servers/jira_mock/       mock Jira MCP server over stdio + fixtures
frontend/src/                api/client.js, hooks/, components/
docs/architecture.md         design rationale
```

PART 2

Architecture and design rationale



The one-sentence version: **a goal is planned into a task DAG by an LLM that writes an agent for each task, inside a retrieved menu of YAML-defined capability envelopes it cannot widen; each agent is created at runtime and granted exactly the MCP tools its envelope allows; the whole thing streams a trace.**

---

1. The idea the design is organised around

A monolithic agent — one big prompt with every tool bolted on — fails in a specific way: adding a capability means editing a prompt, the tool list grows until selection degrades, and there is no point at which you can say *why* a particular tool was used. This project inverts that.

Agents are created, not configured. There is no `JiraAgent` class anywhere in the codebase, and no agent record in YAML either. What `backend/config/agents.yaml` holds is a *capability envelope* — permissions and policy, with no prompt:

```
- id: jira_reporting
  description: >-
    Retrieves and summarizes historical Jira sprint records: committed versus
    completed story points, and the velocity trends that follow from them.
  allowed_tool_selectors: ["jira.get_sprints", "jira.get_sprint_issues"]
  allowed_models: ["claude-sonnet-5", "claude-haiku-4-5"]
  default_model: claude-sonnet-5
  max_iterations_limit: 8
  requires_approval: false
  policy: |
    Never estimate, invent or recall a figure you did not retrieve in this task.
```

The agent itself is written by the planner, per task, at request time:

```
{ "name": "velocity_forecaster",
  "role": "Projects next-sprint velocity from completed points",
  "system_prompt": "Work from actual Jira data, never from assumed numbers.\n\nMethod:\n1. Call
jira.get_sprints …",
  "tool_selectors": ["jira.get_sprints", "jira.get_sprint_issues"],
  "max_iterations": 5 }
```

`app/agents/composer.py` decides whether that agent is allowed to exist, and `app/agents/factory.py` turns the approved spec plus its tools into a coroutine.

Two properties follow, and they are the ones to defend in review:

- **No agent's prompt is a fixed string in this repository.** A `system_prompt` key in `agents.yaml` is rejected at startup — there is a test for it.
- **The set of possible agents is unbounded, not four.** One envelope produces agents that look nothing like each other; in `tests/test_end_to_end.py` the same `jira_reporting` envelope yields one agent with two tools and another with none.

Adding a *capability* is an edit to `agents.yaml`; adding an integration is an edit to `mcp_servers.yaml`. Neither needs Python.

---

2. Agent architecture and routing strategy

Why a planner rather than a router

A classifier that maps a goal to one agent cannot express *"fetch the history, then forecast and check capacity from it in parallel, then review the result."* That is a DAG, not a label. So the routing decision produces a plan.

The routing pipeline

1. **Embed the goal.** `app/embeddings.py`.
2. **Retrieve a menu.** Top-k capabilities ranked by cosine similarity between the goal and each envelope's `description`, top-k tools by their MCP-advertised description, and the top-k most similar *completed* past runs from pgvector. The menu shows each envelope's tools **resolved to concrete names**, because

resolved names are what the planner's request will be checked against — showing it globs would invite a subset it cannot actually have.

3. **One structured-output call.** The planner LLM returns a `TaskPlan`: a list of `{task_id, capability_id, agent, objective, depends_on[]}`, where `agent` is a complete agent it designed for that task.
4. **Validate in two layers.** `validate_plan` in `app/graph/state.py` rejects an unknown `capability_id`, a dangling or self dependency, a duplicate task id, and any dependency cycle. Then `check_envelopes` dry-runs every synthesized agent through `realize_agent`, rejecting any that reaches outside its envelope.
5. **On rejection, retry exactly once**, feeding the error back into the conversation so the model can see which id it invented or which tool it was refused. A second failure fails the run loudly.

The rule that keeps this honest: **the planner designs inside a menu it cannot widen.** It writes the agent, but a capability id that is not in the registry, and a tool outside the envelope, both stop at validation. Tested in `tests/test_planner.py` and `tests/test_composer.py`.

Validating at *planning* time rather than at execution is a deliberate choice: an escalation then costs one retry instead of a run that dies halfway through having already spent money on earlier tasks.

Execution: layered fan-out

The compiled graph has **four nodes regardless of how many agents run**:

```
START → planner → dispatcher ⇄ worker(s) → synthesizer → END
```

`dispatcher` is the scheduler. After the planner and after *every* worker superstep, `route_ready_tasks` recomputes the set of tasks whose dependencies are all satisfied and returns one LangGraph `Send` per task. Independent tasks therefore fan out concurrently, and when nothing is left it routes to the synthesizer instead.

The consequence worth stating out loud: **parallelism is a property of the plan, not of the graph topology.** A plan with four independent tasks runs four workers in one superstep without any change to the graph.

Tool permissions: the envelope

This is the part that makes runtime agent creation safe rather than reckless.

`tool_selectors` are fnmatch globs over namespaced `<server>.<tool>` names. An envelope's `allowed_tool_selectors` is a **ceiling**; a synthesized agent's `tool_selectors` is a **request**. `realize_agent` resolves both through `ToolRegistry.select` and grants the request only if it is a subset:

```
escalation = requested − permitted        # both are sets of resolved tool names
if escalation:
    raise EnvelopeViolationError(...)
```

The comparison is on resolved names, never on the glob strings. That is the whole trick. A synthesized selector of `jira.*` is not textually a subset of an envelope's `jira.get_*`, but resolving both and comparing names is exact. A string-prefix check here would be the bug that lets a generated agent award itself the write tool — `tests/test_composer.py` has that exact case.

Three things are taken from the envelope and never from the agent: `requires_approval` (so a generated agent cannot approve itself), the model allowlist, and the iteration ceiling. The `policy` text is appended *after* the generated prompt under an explicit override header, because later instructions carry more weight — a policy placed first can be talked over by whatever the planner wrote.

The check runs twice: once at planning time so a violation is retryable, and again in the worker, because the worker should not trust a plan payload it was handed.

An empty allowance means no tools — deliberately different from meaning all of them. It is why an `analysis_only` agent physically cannot call Jira, and why a `jira_reporting` agent physically cannot reach the write tool, however it is prompted and whatever it names itself.

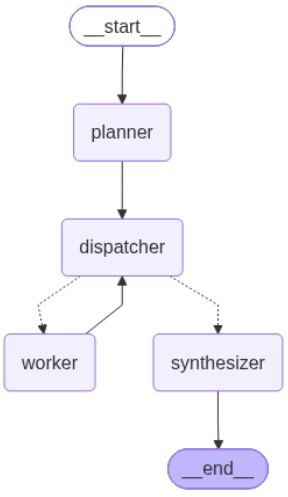
---

3. LangGraph and LangChain — what each is actually used for

Library	Used for	Not used for
LangGraph	The graph runtime: `Send` fan-out, the Postgres checkpointer, and `interrupt()` for human approval	Anything prebuilt. `create_react_agent` is deliberately not used
LangChain	Two narrow things: `langchain-mcp-adapters` to turn MCP tools into bindable tools, and `ChatAnthropic` as the model client	Chains, agents, memory, retrievers

The compiled graph

The entire topology, which never changes no matter what the planner decides:



The compiled graph: four nodes, whatever the plan asks for. The dotted edges out of `dispatcher` are the conditional ones.

Two details are worth reading off it. The dotted edges out of `dispatcher` are the **conditional** ones — that single decision point is the scheduler, and it is what turns a plan of any shape into the right number of parallel workers. And `worker → dispatcher` is a real cycle: every finished task sends control back to be re-evaluated. Six nodes on the page, any number of agents at runtime.

Three LangGraph features carry real weight and are the reason it was chosen over hand-rolling an executor:

- `Send` gives dynamic map-reduce fan-out. Without it, a runtime-decided number of parallel branches means writing a scheduler.
- `interrupt()` + **checkpointer** makes human approval *durable*. The run pauses in Postgres, not in memory, so the process can restart and the run is still resumable. This is the difference between an approval gate and a modal dialog.
- **Reducers on state** make concurrent worker writes safe by construction.

The worker's tool loop is hand-written (`app/agents/factory.py`) rather than delegated to a prebuilt ReAct agent, for three reasons: `max_iterations` is enforced per agent spec, every tool call emits `tool.called` / `tool.result` for the trace, and token usage is attributed per call. A prebuilt agent hides all three inside a subgraph.

LangFlow: considered and rejected

LangFlow's flow definition lives in GUI state. This project's entire premise is that routing is defined by version-controlled configuration and decided at runtime by a planner. A GUI-authored static flow contradicts both halves. It would also make the "add an agent without writing code" property *worse*, not better — a YAML diff reviews; a serialized canvas does not.

Redis: considered and rejected

The event bus is in-process. A single API process has one publisher and a handful of SSE subscribers, so there is nothing for a broker to do. Adding Redis would put a box on this diagram that nothing justifies. The cost is stated plainly in the limitations: the live bus does not fan out across replicas. The durable copy of every trace is in Postgres, which is what makes that acceptable.

---

4. MCP integration

Discovery is at startup, and schemas are never hardcoded. `McpManager` reads `mcp_servers.yaml`, opens a session per server, and calls `list_tools()`. Whatever comes back *is* the platform's tool surface. `GET /tools` returns exactly that, which is the evidence.

Sessions are held open, each inside its own task

This is the one genuinely subtle piece of the backend and it is worth being able to explain. A stdio MCP server is a subprocess; reconnecting per tool call would pay process-spawn cost on every use. But the MCP stdio client is built on anyio task groups, and **an anyio cancel scope must be exited by the task that entered it** — while a web server runs lifespan startup and lifespan shutdown in *different* tasks. Opening the session at startup and closing it at shutdown therefore fails with `attempted to exit cancel scope in a different task`.

The fix is to give each session a dedicated task that opens it, publishes its tools, waits on a stop event, and closes it — both ends in one task.

Policy

`call_tool_with_policy` applies a configurable timeout and bounded retries with exponential backoff. Retries are for *transport* failures and timeouts only. A tool that returns an error result has answered the question, so retrying it just spends the same money for the same answer; the result goes back to the agent, which decides what to do about it.

Failure is graded, not binary:

Failure	Consequence
A server won't start	Reported; its tools are absent; the rest of the platform runs
A tool call exhausts retries	Returned to the model as an error `ToolMessage`; the agent can recover
An agent exceeds `max_iterations`	Task marked `failed`; siblings keep their work; the synthesizer reports the gap
The planner can't produce a valid plan	The run fails loudly — there is nothing to degrade to

Swapping the mock for real Jira

`mcp_servers/jira_mock/` exists because the assignment has no Jira credentials. Replacing it is a config edit — the commented block in `mcp_servers.yaml` shows the shape. No application code refers to Jira by name.

---

5. Memory design and persistence

One Postgres instance, three tiers with genuinely different jobs.

Tier	Store	Written	Read	Job
Working	LangGraph Postgres checkpointer (`thread_id = run_id`)	Every superstep	On resume	Resume a run paused at an approval, across a restart
Episodic	`runs` + `steps` tables	As each event is published	`GET /runs/{id}`	Trace replay and cost history after the in-process bus has forgotten
Semantic	`goal_embedding vector(384)` on `runs`	At run start	At plan time	Inject similar completed runs into the planner prompt

Every event goes to **both** the SSE bus and the `steps` table in the same call (`build_event_recorder`). A trace that exists in one but not the other is worse than having neither.

The embedding model, stated honestly

`app/embeddings.py` uses **OpenAI** `text-embedding-3-small`, with a local lexical vectorizer as a fallback when no key is configured. Three things about that are worth defending.

Why a hosted model at all. The original implementation was a signed hashing vectorizer. Signed hashing is mathematically sound — the ± sign makes the inner product unbiased — but it was badly undersized here: the capability descriptions produce ~275 features into 384 buckets, of which **23–29% collide**, leaving an error of ±0.03–0.08 on similarities of only 0.05–0.25. Measured against exact unhashed cosine over the same features it put the **wrong capability first on four goals out of five**, including the flagship velocity goal. Raising the dimension does not rescue it: at 4096 it still recovered only 3/5. The deeper problem is that it compares spelling, not meaning.

Why 384 dimensions. The model is natively 1536, but the v3 models accept a `dimensions` parameter. Requesting 384 keeps stored vectors the same width as the existing `Vector(384)` column, so the swap needed **no schema change and no migration**. One wrinkle worth knowing: OpenAI normalizes the *full-width* vector, so a truncated one is no longer unit length, and `cosine_similarity` is a bare dot product. `app/embeddings.py` re-normalizes; without that it would silently rank by magnitude as well as direction.

Why the fallback is not decoration. Both registries embed their descriptions when they are constructed, and the test fixtures construct them — so without a working offline path the whole suite would make live API calls on every run. The fallback keeps tests hermetic and offline development possible, at a measured cost: on reworded queries, top-1 retrieval is **5/6 hosted against 3/6 lexical**.

Anthropic has no embeddings API, so semantic retrieval necessarily means a second provider.

One consequence is worth internalising because it is a real lesson of the design: **a capability's `description` is retrieval surface, not documentation.** An early version of the forecasting envelope ranked fourth for a velocity goal because its text never used the word "velocity". Fixing the *config* fixed the routing.

---

6. Cost tracking

Every LLM response's `usage_metadata` is priced against a table of per-MTok rates in `app/costs.py` and accumulated in graph state as one record per call, tagged with the task and agent that spent it. An unpriced model is recorded in `unpriced_models` rather than silently costed at zero. Per-run totals land on the `runs` row and in the `run.completed` event.

---

7. The event contract

Seven names, fixed, and nothing else:

```
run.started   task.started   tool.called   tool.result
approval.required   run.completed   run.failed
```

There is deliberately no `task.completed`. The frontend therefore *derives* per-task completion (`frontend/src/hooks/runState.js`): a task is done when a task that depends on it starts — which is sound because the dispatcher only releases a task once all its dependencies have returned — or when `run.completed` arrives carrying each task's final status.

This is a real trade-off and worth naming: it keeps the wire contract minimal and the client slightly cleverer. An eighth event name would make the client dumber. If this contract were reopened, `task.completed` is the change I would argue for.

---

8. Notable decisions

Failure inside a run is graded rather than fatal. A worker that fails records a failed task; siblings keep their work and the synthesizer is instructed to report what is missing rather than paper over it. A partial answer that says what it is missing beats a 500.

Approval interrupts are detected in the orchestrator, not the worker. A node that emitted its own `approval.required` would emit it a second time, because LangGraph replays the node when the graph resumes.

The synthesizer has no tools, by design. It can only use figures the workers retrieved, and is instructed to say when something was not retrieved. Giving it tools would let it quietly paper over a failed task.

Modules that aren't in the assignment's suggested layout. Four small ones were added rather than growing others past the ~200-line limit: `embeddings.py`, `costs.py`, `chat_models.py` (a single-seam model constructor), and `orchestrator.py` (the run driver, which is genuinely its own job and would otherwise bloat the route handlers).

---

9. What I would change for production

Area	Now	Production
Embeddings	`text-embedding-3-small` at 384 dims, lexical fallback	Drop the fallback, embed asynchronously, and re-embed stored runs on a model change
Event bus	In-process	Postgres `LISTEN/NOTIFY` before reaching for a broker
Schema	`create_all` at boot	Alembic migrations
Auth	None	Per-tenant auth; MCP credentials scoped per user, not per process
Planner quality	Untested	A labelled goal→plan eval set, scored on agent selection
Approval	Any caller can approve	Approver identity recorded and authorised
Forecasting	Rolling average	An actual model — and the honest note that the rolling average is a baseline, not a forecast